TECHNICAL ARCHITECTURE & ENGINEERING REFERENCE

Tracmio 2.5.2 Offline Edition — Complete Developer & Architecture Guide

End-to-end engineering documentation of every Kotlin class, Room entity, SQL migration, repository, paise calculator, backup/export engine, Jetpack Compose screen, WebView bridge, and JavaScript function in devilcharan1613/Fintrack, including a full diff analysis from v2.4.2 to v2.5.2 (PR #3).

Build Attribute	Baseline Build (v2.4.2 APK)	Reviewed Build (v2.5.2 Offline Edition)
Artifact & Branch	`app-debug.apk` (`main` branch)	`FinTrack-2.5.2-debug.apk` (`fintrack-2.5.0-development`)
Application ID	`com.charan.fintrack`	`com.charan.fintrack`
Version & SDKs	v2.4.2 (`versionCode 26`), SDK 23–35	v2.5.2 (`versionCode 29`), SDK 23–35
Reviewed Commit & CI	Pre-2.5.0 baseline state	`60d7f819cbde...d4615` (Run 42745161349)
Room Schema & Size	Schema v3 (8 tables) • 171 classes, 1,105 methods	Schema v6 (9 tables) • 741 classes, 4,278 methods (18,683,558 B)
SHA-256 Digest	Baseline local debug package	`05a917f5d3fef93fec5cfe18d900552bf000f10245b7d95e41542e3f3a80c19f`

> **Architectural Scope & Edition Design**
> FinTrack 2.5.2 is a strictly offline Android personal finance and ledger application with zero network permissions, no login/OTP, and no cloud sync. It ships a dual-runtime architecture: a legacy-compatible WebView shell (`MainActivity + assets/index.html`) that safely migrates existing local storage into Room SQLite v6, and a full native Jetpack Compose UI (`NativeFinTrackActivity + NativeFinTrackApp`) backed by exact 64-bit integer paise arithmetic (`MoneyPaise`).

Contents Overview

Section	Engineering Topic	Focus Area
Section 1	System Architecture, Manifest Configuration & Dual-Stack Runtime	Core Architecture
Section 2	Complete Change Log & Engineering Fixes (v2.4.2 to v2.5.2 / PR #3)	Applied Changes

Section	Engineering Topic	Focus Area
Section 3	Room Database v6 Schema, Entities, Migrations & DAO SQL Reference	Data Persistence
Section 4	Domain Calculators, Repositories & Backup / Export Engines	Business Logic
Section 5	Native Jetpack Compose UI Screens, Dialogs, Navigation & ViewModels	Native Compose UI
Section 6	Hybrid WebView Bridge, Local Reminders & JavaScript Function Reference	WebView & JS
Section 7	QA Verification, Automated Test Suite & Release Readiness Checklist	QA & Release

1. System Architecture, Manifest Configuration & Dual-Stack Runtime

The FinTrack repository (PR #3 on branch `fintrack-2.5.0-development`) transitions the application from a hybrid WebView-centric design in v2.4.2 to a Room-backed, paise-accurate offline architecture in v2.5.2. To protect existing users upgrading from v2.4.x without data loss, the app preserves `MainActivity` as the launcher entry point to run a guarded WebView-to-Room migration before operating alongside the native Jetpack Compose stack (`NativeFinTrackActivity`).

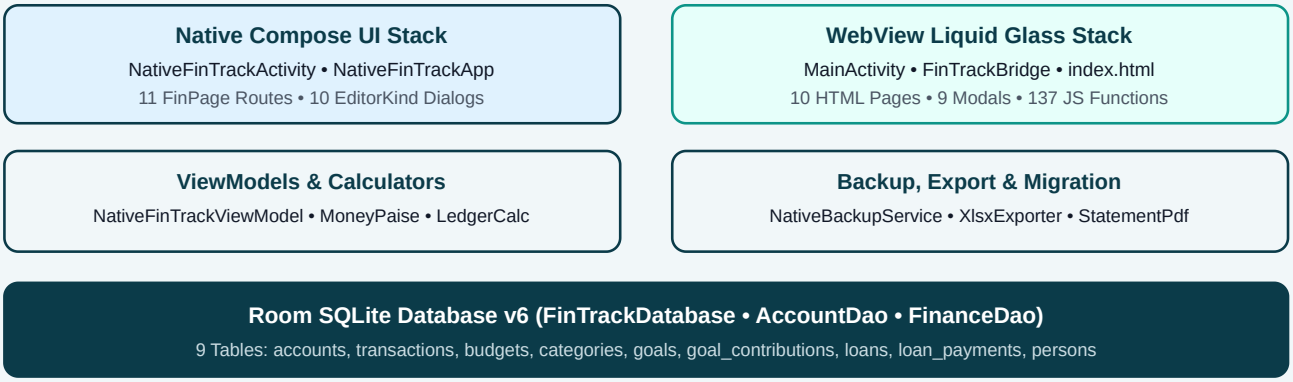


Figure 1: Tracmio 2.5.2 Offline Edition layered architecture across Compose, WebView, Repositories, and Room v6.

AndroidManifest.xml Component & Permission Diff (v2.4.2 vs. v2.5.2)

Manifest Element	v2.4.2 (`app-debug.apk`)	v2.5.2 (`FinTrack-2.5.2-debug.apk`)
Permissions	`POST_NOTIFICATIONS`, `RECEIVE_BOOT_COMPLETED`, `WAKE_LOCK`, `ACCESS_NETWORK_STATE`, `FOREGROUND_SERVICE`	`POST_NOTIFICATIONS`, `RECEIVE_BOOT_COMPLETED`, `REQUEST_INSTALL_PACKAGES` (zero network permissions)

Manifest Element	v2.4.2 (`app-debug.apk`)	v2.5.2 (`FinTrack-2.5.2-debug.apk`)
Activities	`MainActivity` (launcher, `configChanges=1184`)	`MainActivity` (launcher, `launchMode=singleTop`), `NativeFinTrackActivity` (`exported=false`)
Receivers	`ReminderActionReceiver`, `ReminderBootReceiver`, `ReminderTimeChangeReceiver` + WorkManager proxies	`DailyReminderReceiver` (`exported=false`), `ReminderBootReceiver` (`BOOT_COMPLETED`)
Services	WorkManager `SystemAlarmService`, `SystemJobService`, `SystemForegroundService`, Room invalidation	`androidx.room.MultiInstanceInvalidationService` only (WorkManager removed)

2. Complete Change Log & Engineering Fixes (v2.4.2 to v2.5.2 / PR #3)

Below is the complete breakdown of the engineering changes implemented on branch `fintrack-2.5.0-development` (PR #3) and verified in GitHub Actions build run 42745161349.

2.1 Finance & Ledger Integrity Changes

- **Signed 64-Bit Paise Arithmetic (`MoneyPaise`):** Replaced floating-point rupee accumulation with a value class wrapping a signed 64-bit integer (`paise: Long`), using `BigDecimal` `HALF_UP` rounding when converting legacy rupee doubles via `fromLegacyRupees` and `fromLegacySignedRupees`.
- **Atomic Transfer Conversion on Edit (`replaceTransaction`):** Editing an existing `INCOME`, `EXPENSE`, `BORROW`, or `LEND` transaction into a `TRANSFER` now executes inside a single Room transaction that deletes the original row (or old transfer pair via `deleteTransfer`) and inserts a matched `TRANSFER_OUT` and `TRANSFER_IN` pair sharing a common `transferId` and `stableUid` suffix (`:in`).
- **Dual-Account Transfer Filtering with Single Logical Row:** In `NativeFinTrackApp.kt` (`transactionMatchesAccount` and `TransactionsScreen`), visible transaction lists filter out `TRANSFER_IN` rows so each transfer displays once logically, while matching the account filter if either `transaction.accountId == filterId` OR `(transaction.type == "TRANSFER_OUT" && transaction.transferAccountId == filterId)`.
- **Strict Person Association for Borrow/Lend Only:** Previously, the transaction editor's hidden default person state could attach an unrelated `personId` to ordinary `INCOME` or `EXPENSE` rows. Both `NativeFinanceRepository` and `NativeFinTrackViewModel` now retain `personId` strictly when `type.uppercase() in setOf("BORROW", "LEND")` and set `personId = null` otherwise.
- **Room Schema Migration v5 to v6 (`MIGRATION_5_6`):** Cleans existing databases on upgrade by executing: `UPDATE transactions SET personId = NULL WHERE personId IS NOT NULL AND UPPER(type) NOT IN ('BORROW', 'LEND')`.

2.2 Backup & Export Integrity Changes

- **Transactional Snapshot Reads (`ExportSnapshot`):** Both `NativeBackupService.exportJson` and `NativeFinTrackViewModel.exportPdf` read all 9 tables inside a single `database.withTransaction` snapshot so concurrent edits never produce torn backups or statements.
- **Schema Version Gate & Epoch Millisecond Preservation:** `NativeBackupService.mergeJson` validates `format == "FinTrackNativeBackup"` and rejects missing or unsupported future `schemaVersion` values (supporting versions `1..2`) before touching Room. Native restore preserves exact `Long` epoch timestamps down to the millisecond.
- **Matched Transfer Pair Validation (`validateNativeTransferPairs`):** Before inserting records during restore, `NativeBackupService` groups transfer rows by `transferId` and verifies that every transfer has exactly one `TRANSFER_OUT` and one `TRANSFER_IN` record with matching positive paise amounts and consistent cross-referenced source/destination account IDs.
- **Exact Paise XLSX Summary Arithmetic (`XlsxExporter`):** Replaced floating-point sum accumulation in `XlsxExporter.buildSummaryRows` with `exactAmountPaise(JSONObject)` so Summary sheet totals (`Income INR`, `Expense INR`, `Net savings INR`) are computed in exact paise and converted via `MoneyPaise`. Added custom date-range XLSX export (`exportXlsxRange`).
- **CSV Formula Injection Protection:** `NativeBackupService.csv(String)` quotes all CSV fields, escapes internal quotes as `""`, and neutralizes spreadsheet formula injection (leading `=`, `+`, `-`, `@`, tab/CR) on user-supplied text.

2.3 Migration, Data Safety & UI/Device Changes

> **Tracmio Rebranding, 3D Animated Welcome Screen, Multi-Theme Reports & Custom Background Photo Fixes**
>
> **0. Rebranded to "Tracmio — Finance & Expenses Tracker" + 3D Animated Welcome Screen:** Embedded the 3D Liquid Glass `Tracmio` logo (`76767.png`) in the topbar and added `#tracmioWelcomeOverlay` (`openTracmioWelcome()` / `completeTracmioWelcome()`) with 3D levitation (`@keyframes tracmioLogoFloat`), pulsing sonar rings (`@keyframes tracmioSonarPulse`), specular shimmer sweep, personalized user name greeting, and currency onboarding.
>
> **0b. Multi-Style & Multi-Theme Reports + Clean 5-Tab Dock & Custom Background Photo Fix:** Added 4 selectable Report chart styles (`neon_bars`, `cyber_wave`, `capsules`, `minimal` via `setReportVisualStyle()`) and 4 Report color palettes (`aurora`, `tracmio`, `sunset`, `sapphire` via `setReportThemePalette()`). Removed `Reports` from the bottom navigation dock (restoring the 5-tab `Home | Accounts | + | Transactions | More` dock), removed the test meme block from Settings UI (keeping meme notifications documented in Section 6.1), and added `#customBgLayer` before `localStorage` persistence so custom background photos always appear behind translucent frosted glass cards even on `content://` URLs.
>
> **1. Restored 5-Card Transaction Type Grid (`#txnTypeGrid`):** Fixed missing `initFinTrackTheme()` in `assets/index.html` that caused `init()` to throw a `ReferenceError` before executing `buildTypeGrid()`. Added a fallback check inside `openModal('txn')` so the 5-card `Income / Expense / Transfer / Borrow / Lend` grid always renders.
>
> **2. 3D Neon Liquid Glass UI Overhaul:** Upgraded `#shell`, `.hero` (with centered circular `↓` Income and `↑` Expense badges), `.qs-card` (3D glowing squircle SVGs and cyan/violet corner highlights), dashed `.add-acc-mini`, and floating pill `.bottomnav`.
>
> **3. App UI Size Scaling (Increase / Decrease):** Replaced icon-only sizing with `setFinTrackUiSize(value)` and `stepFinTrackUiSize(-5/+5)` (`80%–130%`) scaling `#shell` zoom and `--ft-ui-scale` across the entire app.
>
> **4. Runtime Notifications & Media Permissions:** Added `POST_NOTIFICATIONS` and `READ_MEDIA_IMAGES` / `READ_EXTERNAL_STORAGE` runtime permission requests via `ActivityResultContracts.RequestMultiplePermissions` in `MainActivity.kt` and `requestFinTrackPermissions()` in `assets/index.html`.

- **Corrupted WebView Storage Guard (`LegacyMigrationGuard`):** In `assets/index.html`, if `localStorage.getItem('fintrack_v2_data')` fails JSON parsing or lacks arrays, `preserveUnreadableState(raw)` copies the raw payload to `fintrack_v2_unreadable_backup` and sets `window.stateLoadWarning = true`. When `MainActivity.requestNativeMigrationFromLegacyWebView()` runs, it passes `recoveryRequired: window.stateLoadWarning === true` to `LegacyMigrationGuard.requireReadable(Boolean)`, which aborts migration instead of overwriting Room with an empty seed state.
- **Explicit Non-Destructive Room Migrations:** `FinTrackDatabase` registers `MIGRATION_1_2` through `MIGRATION_5_6` without `fallbackToDestructiveMigration()`.

- **Private Background Photo & Cache-Busting Revision Key:** Selected background images are decoded, compressed to JPEG in app-private storage (`fintrack_background.jpg` via atomic temp file `fintrack_background.tmp`), and tracked with `backgroundImageRevision: MutableIntState` so Jetpack Compose reloads the `Bitmap` immediately even when the file URI path is unchanged.
- **Safe Drawing & Keyboard (IME) Insets:** The native Compose scaffold applies safe drawing and IME insets, automatically hiding `NativeBottomBar` whenever `keyboardVisible` is true.
- **AlarmManager Daily Reminders:** Replaced heavy WorkManager jobs with `FinTrackReminderScheduler`, `DailyReminderReceiver`, and `ReminderBootReceiver`, supporting standard and fun ("meme") reminder messages and notification quick actions (`Add expense`, `Add income`).

3. Room Database v6 Schema, Entities, Migrations & DAO SQL Reference

`FinTrackDatabase` (database name `"fintrack_database"`, Room version `6`, identity hash `9555f638f182d4295cd8819ac2cc4432`) exposes two DAOs: `accountDao(): AccountDao` and `financeDao(): FinanceDao`. Every table in schema v6 carries a unique index on `stableUid` to guarantee idempotent backup merges.

3.1 Complete Room Entity Schema Reference (All 9 Entities)

Entity / Table	Fields & Kotlin Types	Indexes & Defaults
AccountEntity `accounts`	`id: Long` (PK), `name: String`, `accountType: String`, `balance: Double`, `currency: String`, `institution: String`, `accountNumber: String`, `notes: String`, `isArchived: Boolean`, `createdAt: Long`, `updatedAt: Long`, `creditLimit: Double`, `creditLimitPaise: Long`, `stableUid: String`, `openingBalancePaise: Long`	Unique index on `stableUid`. Default `currency = "INR"`, `creditLimitPaise = 0L`, `openingBalancePaise = 0L`.
TransactionEntity `transactions`	`id: Long` (PK), `accountId: Long`, `type: String`, `amount: Double`, `category: String`, `note: String`, `timestamp: Long`, `transferAccountId: Long?`, `transferId: String?`, `subcategory: String`, `personId: Long?`, `stableUid: String`, `amountPaise: Long`, `paymentMethod: String`, `upiApp: String`	Unique index on `stableUid`. Types: `INCOME`, `EXPENSE`, `TRANSFER_OUT`, `TRANSFER_IN`, `BORROW`, `LEND`.
BudgetEntity `budgets`	`id: Long` (PK), `category: String`, `limitAmount: Double`, `month: String`, `stableUid: String`, `limitAmountPaise: Long`, `period: String`	Unique index on `stableUid`. Default `period = "Monthly"` (Daily, Weekly, Monthly, Yearly).

Entity / Table	Fields & Kotlin Types	Indexes & Defaults
CategoryEntity `categories`	`id: Long` (PK), `name: String`, `type: String`, `parentId: Long?`, `icon: String`, `isSystem: Boolean`, `stableUid: String`, `subcategoriesJson: String`	Unique index on `stableUid`. Default `subcategoriesJson = "[]"`. Types: `EXPENSE`, `INCOME`.
GoalEntity `goals`	`id: Long` (PK), `name: String`, `targetAmount: Double`, `savedAmount: Double`, `deadline: Long?`, `stableUid: String`, `targetAmountPaise: Long`, `savedAmountPaise: Long`	Unique index on `stableUid`. Tracks target vs. accumulated paise.
GoalContributionEntity `goal_contributions`	`id: Long` (PK), `goalId: Long`, `amount: Double`, `timestamp: Long`, `note: String`, `stableUid: String`, `amountPaise: Long`	Unique index on `stableUid`; composite index on `(goalId, timestamp)`. Added in v5.
LoanEntity `loans`	`id: Long` (PK), `name: String`, `principal: Double`, `paidAmount: Double`, `monthlyPayment: Double`, `interestRate: Double`, `stableUid: String`, `principalPaise: Long`, `paidAmountPaise: Long`, `monthlyPaymentPaise: Long`	Unique index on `stableUid`. Stores principal, repaid amount, EMI, and annual interest %.
LoanPaymentEntity `loan_payments`	`id: Long` (PK), `loanId: Long`, `amount: Double`, `timestamp: Long`, `note: String`, `stableUid: String`, `amountPaise: Long`	Unique index on `stableUid`. Tracks individual loan repayment records.
PersonEntity `persons`	`id: Long` (PK), `name: String`, `phone: String`, `notes: String`, `stableUid: String`	Unique index on `stableUid`. Linked to `BORROW` and `LEND` transactions.

3.2 Room Database Migrations & Companion Helpers (`FinTrackDatabase.kt`)

Migration / Helper	Exact SQL & Migration Behavior
`MIGRATION_1_2`	Creates `transactions`, `budgets`, `goals`, and `loans` tables alongside initial `accounts` table.
`MIGRATION_2_3`	Adds `subcategory` and `personId` to `transactions`; creates `categories`, `persons`, and `loan_payments` tables.
`MIGRATION_3_4`	Adds `stableUid` and `*Paise` integer columns across all 8 tables, runs `backfillSignedMoney` (for `accounts.balance` to `openingBalancePaise`), `backfillMoney` (for positive amounts), and `backfillStableUids`, then creates unique indexes on `stableUid`.
`MIGRATION_4_5`	Adds `creditLimit` & `creditLimitPaise` to `accounts`; `paymentMethod` & `upiApp` to `transactions`; `period` to `budgets`; `subcategoriesJson` to `categories`; creates `goal_contributions` table and indexes.

Migration / Helper	Exact SQL & Migration Behavior
`MIGRATION_5_6`	Executes `UPDATE transactions SET personId = NULL WHERE personId IS NOT NULL AND UPPER(type) NOT IN ('BORROW', 'LEND')` to purge invalid person links.
`Companion` Helpers	`getDatabase(Context)` (singleton builder with `ALL_MIGRATIONS`), `addColumn(...)`, `backfillMoney(...)`, `backfillSignedMoney(...)`, and `backfillStableUids(...)`.

3.3 AccountDao & FinanceDao Method & SQL Query Reference

DAO Method Signature	Return Type	Underlying Room SQL Query / Operation
`AccountDao.getAllAccounts()`	`Flow<List>`	`SELECT * FROM accounts WHERE isArchived = 0 ORDER BY createdAt DESC`
`AccountDao.allAccounts()`	`suspend List`	`SELECT * FROM accounts` (includes archived accounts for backup/validation)
`AccountDao.getAccountById(id)`	`suspend Entity?`	`SELECT * FROM accounts WHERE id = ? LIMIT 1`
`AccountDao.accountByStableUid(uid)`	`suspend Entity?`	`SELECT * FROM accounts WHERE stableUid = ? LIMIT 1`
`AccountDao.getBalance(id)`	`Flow<Double>`	`SELECT COALESCE(SUM(a.balance), 0) + COALESCE((SELECT SUM(CASE WHEN t.type IN ('INCOME','TRANSFER_IN','BORROW') THEN t.amount ELSE -t.amount END) FROM transactions t WHERE t.accountId = a.id), 0) FROM accounts a WHERE a.id = ? AND a.isArchived = 0`
`AccountDao.getComputedTotalBalance()`	`Flow<Double>`	Computes sum of opening `a.balance` plus signed transaction deltas across all non-archived accounts.
`AccountDao.getTotalBalance()`	`Flow<Double?>`	`SELECT SUM(balance) FROM accounts WHERE isArchived = 0`
`AccountDao.archiveAccount(id, ts)`	`suspend Unit`	`UPDATE accounts SET isArchived = 1, updatedAt = ? WHERE id = ?`
`AccountDao.insert / update / delete / deleteAll`	`suspend Long / Unit`	`INSERT OR REPLACE INTO accounts`, `UPDATE OR ABORT accounts`, `DELETE FROM accounts WHERE id = ?`, `DELETE FROM accounts`.
`FinanceDao.transactions() / allTransactions()`	`Flow / suspend List`	`SELECT * FROM transactions ORDER BY timestamp DESC` (Flow) and `SELECT * FROM transactions` (snapshot).
`FinanceDao.budgets() / allBudgets()`	`Flow / suspend List`	`SELECT * FROM budgets ORDER BY category` and `SELECT * FROM budgets`.

DAO Method Signature	Return Type	Underlying Room SQL Query / Operation
`FinanceDao.categories()` / `allCategories()`	`Flow` / `suspend List`	`SELECT * FROM categories ORDER BY type, name` and `SELECT * FROM categories`.
`FinanceDao.goals()` / `allGoals()`	`Flow` / `suspend List`	`SELECT * FROM goals ORDER BY id DESC` and `SELECT * FROM goals`.
`FinanceDao.goalContributions(id)` / `allGoalContributions()`	`Flow` / `suspend List`	`SELECT * FROM goal_contributions WHERE goalId = ? ORDER BY timestamp DESC` and `SELECT * FROM goal_contributions`.
`FinanceDao.loans()` / `allLoans()`	`Flow` / `suspend List`	`SELECT * FROM loans ORDER BY id DESC` and `SELECT * FROM loans`.
`FinanceDao.loanPayments(id)` / `allLoanPayments()`	`Flow` / `suspend List`	`SELECT * FROM loan_payments WHERE loanId = ? ORDER BY timestamp DESC` and `SELECT * FROM loan_payments`.
`FinanceDao.persons()` / `allPersons()`	`Flow` / `suspend List`	`SELECT * FROM persons ORDER BY name` and `SELECT * FROM persons`.
`FinanceDao.*ByStableUid(uid)`	`suspend Entity?`	`SELECT * FROM <table> WHERE stableUid = ? LIMIT 1` for all 8 finance tables.
`FinanceDao.deleteTransfer(transferId)`	`suspend Unit`	`DELETE FROM transactions WHERE transferId = ?` (removes both `TRANSFER_OUT` and `TRANSFER_IN` rows).
`FinanceDao.spentForCategory / totalIncome / totalExpense`	`Flow<Double>`	Sums `amount` for `('EXPENSE','LEND')` (with timestamp range for category) or `('INCOME','BORROW')`.
`FinanceDao.insert* / update* / delete* / deleteAll*`	`suspend Long / Unit`	Full CRUD adapters for all 8 finance tables (`insertGoalContribution` uses `INSERT OR IGNORE`; all others use `INSERT OR REPLACE`).

4. Domain Calculators, Repositories & Backup / Export Engines

4.1 Money, Date-Time, Migration Guard & Ledger Calculators

Class / File	Functions & Signatures	Behavior & Validation Rules
MoneyPaise `MoneyPaise.kt`	`fromRupees(str)`, `fromSignedRupees(str)`, `fromLegacyRupees(dbl)`, `fromLegacySignedRupees(dbl)`, `fromSignedPaise(long)`, `plus(other)`, `minus(other)`, `toRupeesDecimal()`, `formatInr(locale)`	Value class wrapping `paise: Long`. Rejects negative inputs in unsigned factories; converts rupees via `BigDecimal HALF_UP`; formats currency using `Locale("en", "IN")`.

Class / File	Functions & Signatures	Behavior & Validation Rules
TransactionDateTimeKt `TransactionDateTime.kt`	`parseTransactionDateTime(date: String, time: String = "12:00"): Long`	Parses `yyyy-MM-dd HH:mm` in local timezone into epoch milliseconds; throws `IllegalArgumentException` on malformed dates.
LegacyMigrationGuard `LegacyMigrationGuard.kt`	`requireReadable(recoveryRequired: Boolean)`	Throws descriptive error stopping native migration if `recoveryRequired` is true so corrupted WebView data is never replaced by seed data.
NativeLedgerCalculator `NativeLedgerCalculator.kt`	`signedDeltaPaise(txn)`, `accountBalancePaise(acc, txns)`, `totalBalancePaise(accs, txns)`, `netWorthPaise(accs, txns)`, `incomePaise(txns)`, `expensePaise(txns)`, `borrowedPaise(txns)`, `lentPaise(txns)`, `categoryExpensePaise(...)`	Positive delta for `INCOME`, `BORROW`, `TRANSFER_IN`; negative delta for `EXPENSE`, `LEND`, `TRANSFER_OUT`. `accountBalancePaise = openingBalancePaise + sum(signedDeltaPaise)`. `netWorthPaise = totalBalancePaise + lentPaise - borrowedPaise`.
FinancialCalculator `FinancialCalculator.kt`	`balance(acc, txns)`, `categoryTotal(txns, cat, from, to)`, `monthlyEmi(p, rate, months)`, `netCashFlow(...)`, `progress(cur, target)`	Double-precision calculator retained for legacy unit tests and EMI amortization computation.
CsvParserKt `CsvParser.kt`	`parseCsvLine(line: String): List<String>`	Parses RFC-4180 quoted CSV rows, unescaping doubled quotes (`""`).

4.2 Repositories (`NativeFinanceRepository`, `FinanceRepository`, `AccountRepository`)

`NativeFinanceRepository` is the primary repository for the v2.5.2 Native Compose application. It combines `accountDao.getAllAccounts()` and `dao.transactions()` into a reactive `dashboard: Flow<NativeDashboardTotals>` (computing `balancePaise`, `incomePaise`, `expensePaise`, `borrowedPaise`, `lentPaise`, `netWorthPaise`, and logical `transactionCount` excluding `TRANSFER_IN`).

Repository Method	Transactional Logic & Integrity Checks
`addAccount(name, type, opening, creditLimit, inst, number, notes)`	Requires non-blank `name` and `accountType`, and `creditLimit >= 0`. Generates a UUID `stableUid` and inserts `AccountEntity` with both paise and rupee representations.
`updateAccount` / `archiveAccount` / `deleteAccount`	`deleteAccount` checks if any transactions reference `account.id` (as `accountId` or `transferAccountId`); if linked transactions exist, throws `"This account has linked transactions. Archive it instead."`

Repository Method	Transactional Logic & Integrity Checks
`saveTransaction(accountId, type, amount, category, note, ts, subcat, personId, method, upiApp)`	Validates `amount > 0`, non-blank `category`, existing account, and valid type (`INCOME`, `EXPENSE`, `BORROW`, `LEND`). Requires a valid existing `personId` for `BORROW` / `LEND` and clears `personId = null` for `INCOME` / `EXPENSE`.
`addTransfer(fromId, toId, amount, note, ts)`	Requires `fromId != toId`, `amount > 0`, and both accounts to exist. Inside `database.withTransaction`, inserts a `TRANSFER_OUT` row on `fromId` and a matched `TRANSFER_IN` row on `toId` sharing `transferId`.
`replaceTransaction(txn, destinationAccountId)`	Executes inside `database.withTransaction`. If the edited transaction was part of a transfer (or is being converted into a `TRANSFER`), deletes the old row/transfer pair and inserts a fresh matched `TRANSFER_OUT` / `TRANSFER_IN` pair; otherwise updates the single row while enforcing person-link rules.
`deleteTransaction(txn)`	Inside `database.withTransaction`: if `txn.transferId != null`, calls `dao.deleteTransfer(transferId)` to remove both sides of the transfer; otherwise deletes the single row.
`addPerson` / `updatePerson` / `deletePerson`	Requires non-blank person name. `deletePerson` checks whether any transaction links to `person.id` and blocks deletion with `"This person has linked transactions."`
`addCategory` / `updateCategory` / `deleteCategory`	Validates non-blank name and type in `("EXPENSE", "INCOME")`; serializes subcategory list to JSON array string (`subcategoriesJson`).
`addBudget` / `updateBudget` / `deleteBudget`	Validates positive limit, period in `("Daily", "Weekly", "Monthly", "Yearly")`, and `validateYearMonth(month)` matching real calendar `YYYY-MM`.
`addGoal` / `updateGoal` / `deleteGoal` / `addGoalContribution`	`addGoalContribution(goalId, contribution, note)` runs in a Room transaction: verifies `contribution > 0` and `savedAmountPaise + contribution <= targetAmountPaise` (throws `"Contribution exceeds remaining goal amount"` if exceeded), inserts `GoalContributionEntity`, and updates `GoalEntity.savedAmountPaise`. `deleteGoal` cascades to `deleteGoalContributions(goal.id)`.
`addLoan` / `updateLoan` / `deleteLoan` / `addLoanRepayment`	`addLoanRepayment(loanId, amount, note)` runs in a Room transaction: verifies `amount > 0` and `paidAmountPaise + amount <= principalPaise`, inserts `LoanPaymentEntity`, and updates `LoanEntity.paidAmountPaise`. `deleteLoan` cascades to `deleteLoanPayments(loan.id)`.
`resetAllFinanceData()`	Inside `database.withTransaction`, deletes all rows across all 9 tables: `deleteAllTransactions`, `deleteAllGoalContributions`, `deleteAllLoanPayments`, `deleteAllBudgets`, `deleteAllGoals`, `deleteAllLoans`, `deleteAllCategories`, `deleteAllPersons`, and `accountDao.deleteAll()`.

4.3 Backup, Restore & Export Engines (`NativeBackupService`, `XlsxExporter`, `NativeStatementPdf`)

Service / Class	Key Methods	Format & Data Integrity Guarantees
NativeBackupService `NativeBackupService.kt`	`exportJson()`, `mergeJson(json)`, `exportCsv()`, `validateNativeTransferPairs`, `validateDuplicateSourceIds`, `legacyAccountDeltas`	Exports `FinTrackNativeBackup` (`schemaVersion = 2`, `appVersion = "2.5.2"`) from an atomic `ExportSnapshot`. `mergeJson` supports both native backups (v1–v2) and legacy WebView JSON states, deduplicating every entity by `stableUid` and returning a 10-field `BackupMergeReport`.
XlsxExporter `XlsxExporter.kt`	`create(json)`, `buildSummaryRows`, `logicalTransactions`, `buildRows`, `buildRepaymentRows`, `zipWorkbook`	Generates a multi-sheet OpenXML `.xlsx` archive (`Summary`, `Accounts`, `Transactions`, `People`, `Categories`, `Budgets`, `Savings Goals`, `Loans`, `Loan Repayments`) with frozen header panes (`ySplit="1"`), auto-filters, dark teal header styling (`#073746`), and exact paise summary arithmetic.
NativeStatementPdf `NativeStatementPdf.kt`	`create(accounts, transactions): ByteArray`	Renders a paginated A4 PDF statement via Android's native `PdfDocument` Canvas API, including summary cards (Income, Expense, Transactions), formatted date/account/amount columns, logical transfer arrows (`Account A → Account B`), and page footers.

5. Native Jetpack Compose UI Screens, Dialogs, Navigation & ViewModels

The native UI package (`com.charan.fintrack.ui.nativeapp`) is hosted by `NativeFinTrackActivity` and driven by `NativeFinTrackViewModel`. Styling is defined in `FinTrackNativeTheme.kt` using a Liquid Glass palette (`Midnight #04131A`, `DeepNavy #07222F`, `Teal #18F2D0`, `Cyan #38BDF8`, `Violet #A78BFA`, `Ice #EDF7FA`).

5.1 Navigation Routes (`FinPage`) & Modal Editor Modes (`EditorKind`)

11 Navigation Routes (`FinPage` Enum)

Bottom bar routes: `HOME` (`"home"`), `TRANSACTIONS` (`"transactions"`), `ACCOUNTS` (`"accounts"`), `REPORTS` (`"reports"`), `MORE` (`"more"`). Secondary routes: `PEOPLE` (`"people"`), `BUDGETS` (`"budgets"`), `GOALS` (`"goals"`), `LOANS` (`"loans"`), `CATEGORIES` (`"categories"`), `SETTINGS` (`"settings"`).

10 Modal Editor Modes (`EditorKind` Enum)

Controlled by `NativeEditorDialog`: `NONE`, `TRANSACTION`, `ACCOUNT`, `PERSON`, `BUDGET`, `GOAL`, `LOAN`, `CONTRIBUTION`, `REPAYMENT`, and `CATEGORY`. Supports notification quick actions (`"expense"` / `"income"`).

5.2 Complete Jetpack Compose Screen & Component Inventory (`NativeFinTrackApp.kt`)

Composable / Helper	UI Structure, Controls & User Interactions
`NativeFinTrackApp` & `NativeFinTrackContent`	Root state host managing `SharedPreferences("fintrack_native_settings")` for `dark_theme`, `background_uri`, `background_opacity`, and `icon_scale`, plus 7 SAF file launchers (JSON backup open/save, CSV save, XLSX full/range save, PDF save, background image picker, APK installer). Applies safe-drawing and IME insets and renders custom background bitmap overlay.
`NativeHeader` & `NativeBottomBar`	`NativeHeader` displays dynamic time-of-day greeting (`greetingText()`), page title, and a contextual `+ Add` tonal button. `NativeBottomBar` renders 5 tabs plus a center floating `+` action button with user-adjustable `iconScale`, auto-hiding when the soft keyboard is visible.
`DashboardScreen`	Displays hero `GlassCard` with `TOTAL BALANCE`, Income, and Expense; Quick Stats grid (`Borrowed`, `Lent`, `Net Worth`, `Transactions`); horizontal/vertical `ACCOUNTS & WALLETS` cards with live paise balances; and top `RECENT TRANSACTIONS` with a `View all` button.
`TransactionsScreen` & `TransactionItem`	Provides live text search (`Search transactions, notes or amount`), type filter chips (`ALL`, `EXPENSE`, `INCOME`, `TRANSFER`, `BORROW`, `LEND`), account dropdown filter (matching both source and destination accounts on transfers via `transactionMatchesAccount`), sort dropdown (`Newest first`, `Oldest first`, `Highest amount`), and custom `From date` / `To date` filters.
`AccountsScreen`	Lists all active cash, bank, wallet, and credit accounts in `GlassCard` containers showing current computed paise balance, opening balance, institution, account reference, credit limit, and `Edit` / `Archive` actions.
`ReportsScreen`	Features period selector (`This month`, `This week`, `Today`, `This year`, `All time`, `Custom` date range), a native Compose `Canvas` comparative bar chart + cash flow card (`Income`, `Expense`, `Borrowed`, `Lent`, `Net cash flow`), account balance breakdown, sorted category expense breakdown, and per-person Borrowed/Lent summary.

Composable / Helper	UI Structure, Controls & User Interactions
`MoreScreen`	Hub screen with navigation cards for `People`, `Budgets`, `Savings Goals`, `Loans & Repayments`, `Categories`, and `Settings & Appearance`, plus an offline privacy notice card.
`PeopleScreen`	Displays each contact with aggregate `Borrowed` and `Lent` paise totals, count of linked transactions, recent linked transaction items, and `Edit` / `Delete` buttons.
`BudgetsScreen`	Computes period window via `budgetWindow(BudgetEntity)` (Daily, Weekly, Monthly, Yearly), calculates spent vs. limit paise, renders a progress bar with `Remaining` or `Over budget by` status, and provides `Edit` / `Delete` buttons.
`GoalsScreen`	Shows each savings goal's saved vs. target paise, linear progress bar, `Add contribution` button, `Edit` / `Delete` actions, and live history of recorded `GoalContributionEntity` entries.
`LoansScreen`	Displays each loan's `Outstanding` balance, `Principal`, `Paid` amount, EMI, interest rate, progress bar, `Record repayment` button, note that repayment tracking does not alter cash balances, and full repayment history.
`CategoriesScreen`	Lists all income and expense categories with icons, types, parsed subcategory tags from `subcategoriesJson`, and `Edit` / `Delete` actions.
`SettingsScreen`	Grouped into 7 cards: (1) **Appearance** (Dark Liquid Glass vs. Light theme, custom background photo choose/remove, background visibility slider, navigation icon size slider); (2) **Local backup and restore** (Export/Restore JSON); (3) **Export reports** (CSV, Excel workbook, PDF statement); (4) **Date-range Excel export**; (5) **Daily reminders** (enable toggle, time picker, fun reminder messages toggle); (6) **Manual app update** (APK picker); (7) **Offline privacy, App security & Reset finance data** (with confirmation dialog).
`NativeEditorDialog` & Shared Widgets	Unified modal dialog adapting fields to all 10 `EditorKind` values. Shared UI primitives: `GlassCard`, `MoneyField`, `MetricMini`, `SmallMetric`, `SectionLabel`, and `EmptyState`.

6. Hybrid WebView Bridge, Local Reminders & JavaScript Reference

`MainActivity.kt` hosts the WebView runtime (`file:///android_asset/index.html`), registers the `@JavascriptInterface` bridge as `window.FinTrackBridge`, manages SAF file pickers, handles APK package installation, and orchestrates automatic migration from legacy WebView `localStorage` into Room SQLite v6.

6.1 MainActivity, FinTrackBridge & Reminder Receivers (`MainActivity.kt`)

> **Complete Catalog of Meme Notifications, Standard Reminders & Smart Alerts**
>
> FinTrack schedules local offline notifications on channel `fintrack_daily` (`"FinTrack daily reminder"`) via `FinTrackReminderScheduler` and `DailyReminderReceiver` (stored in `fintrack_reminders` SharedPreferences with keys `enabled`, `hour`, `minute`, `meme_mode`, and `last_meme_index`). Every notification includes two interactive quick-action buttons via `ReminderNotificationActionReceiver`:
>
> - **Quick Action 1 — `Add income` (`EXTRA_QUICK_ACTION = "income"`):** Launches `MainActivity` and invokes `window.finTrackHandleNotificationAction('income')`, opening `#overlay-txn` pre-set to `Income`.
> - **Quick Action 2 — `Add expense` (`EXTRA_QUICK_ACTION = "expense"`):** Launches `MainActivity` and invokes `window.finTrackHandleNotificationAction('expense')`, opening `#overlay-txn` pre-set to `Expense`.

Notification Category	Trigger / Mode Flag	Exact Message Strings Sent to User
Meme / Fun Check-In Notifications	`memeMode = true` (rotates via `last_meme_index` at scheduled time, default `20:00`)	• *"Your wallet called. It wants today's transactions logged 💸 Add your income and expenses!"* • *"Money moved? Tell FinTrack about it before you forget 👀 Add a transaction now."* • *"Your budget is waiting for its daily update 📊 Log today's income and expenses."* • *"UPI went 'Ting!' 5 times today 📱 Did FinTrack hear about that Swiggy order?"* • *"Salary came in like a lion and left like a lamb 🦁➡️🐑 Track where it went today!"* • *"Checking bank balance without logging expenses is horror movie behavior 👻 Log now!"*
Standard Check-In Notifications	`memeMode = false` (rotates daily at scheduled reminder time)	• *"Your daily money check-in is ready. Add today's income and expenses to keep your balance accurate."* • *"A one-minute update helps you stay on top of your money. Log today's transactions in FinTrack."* • *"Small daily updates make monthly reports useful. Add any income or expenses you made today."*
Smart Budget, EMI, Goal & Udhaar Alerts	Triggered on threshold cross (`>=80%` / `>100%` budget), EMI due date, or test trigger	• **Budget Warning:** *"⚠️ Budget Alert (74% Used): You've spent ₹18,500 of your ₹25,000 Bills & Utilities monthly budget."* • **Loan / EMI Due:** *"📅 EMI Due Reminder: HDFC Personal Home Upgrade Loan EMI of ₹14,850 is scheduled this month."* • **Goal Milestone:** *"🎯 Goal Milestone: You reached 71.6% (₹2,15,000 / ₹3,00,000) of your 6-Month Emergency Fund!"* • **Udhaar Settlement:** *"🤝 Pending Udhaar Alert: Rahul Sharma owes you ₹12,000. Tap to settle or record repayment."*

Notification Category	Trigger / Mode Flag	Exact Message Strings Sent to User
6-Theme Preset Grid, Custom Theme Studio & Slider Fix	`setFinTrackTheme()`, `applyCustomFinTrackTheme()`, `setFinTrackBackgroundVisibility()`, `setFinTrackUiSize()`	• **6 Built-In Themes + Custom Studio:** `dark` (Theme 1: Aurora Teal), `obsidian` (Theme 2: Obsidian Gold & Emerald), `sapphire` (Theme 3: Sapphire Prism), `light` (Theme 4: VisionOS Arctic Pearl), `ruby` (Theme 5: Cyber Ruby), `emerald` (Theme 6: Matrix Emerald), plus dual color pickers (`--ft-custom-neon-1`, `--ft-custom-neon-2`). • **Live Slider Fix:** Exported all 64 inline event handlers onto `window.*` outside the IIFE and bound direct `input` / `change` listeners on `#backgroundVisibility` (`0%–100%`) and `#appIconSize` (`80%–130%` UI zoom).

Class / Component	Methods & Runtime Responsibilities
MainActivity	`onCreate`, `onNewIntent`, `requestNativeMigrationFromLegacyWebView`, `migrateLegacyPayload`, `persistLegacyPreferences`, `handleSelectedBackgroundImage`, `readBackup`, `writePendingExport`, `printHtmlDocument`, `handleSelectedApk`, `canInstallPackages`, `launchApkInstaller`, `applyReminder`, `notifyReminderPermissionResult`, `flushPendingQuickAction`, `handleWebViewNavigation`.
MainActivity.FinTrackBridge	JS-callable methods: `exportBackup(json, fileName)`, `importBackup()`, `exportCsv(csv, fileName)`, `exportWorkbook(json, fileName)`, `printDocument(html, jobName)`, `chooseBackgroundImage()`, `scheduleDailyReminder(enabled, hour, minute, memeMode)`, and `installApk()`.
FinTrackReminderScheduler & **ReminderTime**	`schedule(context, enabled, hour, minute)` sets or cancels a daily alarm via Android `AlarmManager`; `restoreAfterBoot(context)` reschedules saved alarms from `SharedPreferences("fintrack_reminders")`; `ReminderTime.nextTriggerAt(nowMillis, hour, minute)` computes the next occurrence timestamp.
DailyReminderReceiver & **ReminderBootReceiver**	`DailyReminderReceiver.onReceive` creates notification channel `fintrack_daily`, rotates through standard or meme messages (tracking `last_normal_index` / `last_meme_index`), attaches quick-action intents (`Add expense`, `Add income` via `EXTRA_QUICK_ACTION`), posts the notification, and schedules the next day's alarm.

6.2 WebView Single-Page Application Reference (`assets/index.html` — 3,468 Lines)

The embedded HTML/CSS/JS application in `assets/index.html` implements 10 page views (`#page-dashboard`, `#page-accounts`, `#page-transactions`, `#page-persons`, `#page-budgets`, `#page-goals`, `#page-loans`, `#page-more`, `#page-reports`, `#page-settings`), 9 modal overlays (`#overlay-txn`, `#overlay-quickcat`, `#overlay-quicksubcat`, `#overlay-quickwallet`,

`#overlay-account`, `#overlay-person`, `#overlay-accaction`, `#overlay-persondetail`, `#overlay-planner`), and 137 JavaScript functions organized below:

JS Module	Complete Function Inventory in `assets/index.html`
Core State, Storage & Formatting	`uid()`, `fmt(n)`, `today()`, `localDateISO(date)`, `isValidISODate(value)`, `roundMoney(value)`, `fmtDate(d)`, `fmtDateShort(d)`, `getInitials(name)`, `escHtml(str)`, `getCat(id)`, `getAcc(id)`, `getPerson(id)`, `getTxnType(id)`, `save()`, `preserveUnreadableState(raw)`, `load()`, `buildSeedState()`, `initState()`, `showToast(msg, type)`, `greeting()`, `init()`, `parseNum(str)`, `formatLike(n)`, `wireCountUp(id)`.
Theme, Background & Native Hooks	`setFinTrackTheme(theme)`, `applyFinTrackTheme(theme)`, `setFinTrackBackgroundVisibility(value)`, `setFinTrackIconSize(value)`, `applyFinTrackBackground(dataUrl)`, `setFinTrackBackgroundFromNative(dataUrl)`, `chooseFinTrackBackground()`, `handleFinTrackBackgroundFile(input)`, `removeFinTrackBackground()`, `initFinTrackBackground()`, `window.finTrackApplyInsets`, `window.finTrackHandleBack`, `window.finTrackHandleNotificationAction`, `window.finTrackReminderPermissionResult`.
Navigation, Dashboard & Ledger Totals	`showPage(id)`, `fabAction()`, `renderCurrentPage()`, `sumByType(type)`, `totalBalance()`, `computeTotals()`, `personStats(personId)`, `sum(type)`, `renderDashboard()`, `txnRowHtml(t)`, `renderMore()`.
Transactions & Quick Creation Modals	`applyTxnDatePreset(preset)`, `clearTxnAdvancedFilters()`, `renderTransactions()`, `setTxnFilter(f)`, `openModal(type)`, `closeModal(type)`, `buildTypeGrid()`, `setTxnType(type)`, `populateAccountSelects(selectedFromId, selectedToId)`, `hasAccount(id)`, `handleAccountSelect(selectEl, target)`, `openQuickWallet(target)`, `closeQuickWallet()`, `saveQuickWallet()`, `populateCategorySelect(selectedId)`, `handleCategorySelect(el)`, `populateSubcategories(selectedSubcat)`, `handleSubcategorySelect(el)`, `openQuickCategory()`, `closeQuickCategory()`, `saveQuickCategory()`, `openQuickSubcategory()`, `closeQuickSubcategory()`, `saveQuickSubcategory()`, `populatePersonSelect()`, `resetTxnForm()`, `editTxn(id)`, `applyTxnEffect(t, sign)`, `toggleCustomUpiApp()`, `toggleTxnUpi()`, `saveTxn()`, `deleteTxn()`.
Accounts & People Management	`renderAccounts()`, `resetAccountForm()`, `toggleCreditLimitField()`, `openAccountEdit(id)`, `saveAccount()`, `deleteAccount()`, `openAccAction(accId, action)`, `saveAccAction()`, `renderPersons()`, `resetPersonForm()`, `openPersonEdit(id)`, `savePerson()`, `deletePerson()`, `openPersonDetail(id)`, `editPersonFromDetail()`.

JS Module	Complete Function Inventory in `assets/index.html`
Budgets, Goals & Loans Planner	`plannerCard(name, main, detail, percent, type, id, color)`, `renderBudgets()`, `renderGoals()`, `renderLoans()`, `openPlannerModal(type, id)`, `closePlannerModal()`, `savePlannerItem()`, `addGoalContribution(id)`, `addLoanRepayment(id)`, `viewLoanRepayments(id)`, `editPlannerItem(type, id)`, `deletePlannerItem(type, id)`.
Reports, Charts & Data Export/Settings	`periodRange(period, ref)`, `inRange(dateStr, range)`, `renderReports()`, `setReportPeriod(p)`, `applyReportDateRange()`, `clearReportDateRange()`, `resolveColor(cssVar)`, `drawBarChart()`, `dateKey(date)`, `roundRect(ctx,x,y,w,h,r)`, `drawDoughnutChart(periodTxns)`, `animateCanvas(canvas, draw)`, `renderCatBreakdown(periodTxns)`, `renderPersonSummary()`, `renderAccBalances()`, `renderSettings()`, `loadReminderSettings()`, `saveReminderSettings()`, `exportExcelWorkbook()`, `exportExcelDateRange()`, `chooseOfflineApkUpdate()`, `openExportDialog(source)`, `applyExportPreset(preset)`, `closeExportDialog()`, `finishTransactionExport(format)`, `exportData()`, `importBackupText(text)`, `window.__importBackupFromAndroid`, `triggerImport()`, `resetData()`.

7. QA Verification, Automated Test Suite & Release Readiness Checklist

In GitHub Actions workflow run 42745161349, the CI build job (ID `129595385967`) passed all compilation, static offline-manifest verification, inline JS syntax checks, and unit test suites (28 unit test methods). The Android emulator instrumentation job (ID `129597080293`, covering 27 instrumentation test methods) was still in progress when the QA report was drafted and must be confirmed alongside a physical-device smoke test prior to merging PR #3 into `main`.

Verification Domain	Automated Coverage	Key Assertions & Manual Smoke Test Requirements
Unit Test Suite `testDebugUnitTest`	28 unit tests (**PASSED** in CI Job `129595385967`)	Verifies `MoneyPaise` rounding and arithmetic, `NativeLedgerCalculator` balances, `FinancialCalculator` EMI/cash flow, `TransactionDateTimeKt` parsing, and `LegacyMigrationGuard`.
Instrumentation Suite `assembleDebugAndroidTest`	27 Android tests (Compiled; Emulator Job `129597080293` pending)	Tests Room migrations v1–v6, `NativeFinanceRepository` transfer editing & person-link rules, `NativeBackupService` merge idempotency & schema version rejection, `XlsxExporter` paise totals, `NativeStatementPdf`, background image replacement, and Compose navigation.

Verification Domain	Automated Coverage	Key Assertions & Manual Smoke Test Requirements
Known Architectural Boundaries	By Design (Offline Edition v2.5.2)	Local SQLite database is unencrypted (no SQLCipher or app lock); goal contributions and loan repayments are tracking records that do not auto-deduct cash accounts; clearing app storage deletes local data unless a JSON backup is exported.
Pre-Release Device Smoke Checklist	Required before merging PR #3 to `main`	Verify on a physical Android device: (1) WebView-to-Room migration on upgrade, (2) income/expense-to-transfer editing, (3) JSON/CSV/XLSX/PDF export & restore in external apps, (4) custom background photo replacement, (5) IME keyboard inset behavior, and (6) Android 13+ notification permission prompt & daily reminder alarm.